

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 02 -

PRIMEIROS PASSOS COM AWS LAMBDA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Criar soluções serverless não significa apenas criar aplicações rodando em nuvem. Nesta aula sobre os primeiros passos com AWS lambda, veremos tópicos como a abordagem do AWS Lambda, a configuração do ambiente de execução (Runtime), gerenciamento do ciclo de vida e o AWS Lambda como componente em uma arquitetura cloud. Além disso, abordaremos os desafios da arquitetura e uso do serverless na AWS.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula, daremos os primeiros passos na utilização do serviço AWS Lambda, entendendo como ele funciona, suas vantagens e desafios.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

A computação serverless é um modelo de execução na nuvem no qual um provedor, como a AWS, gerencia dinamicamente a alocação de máquinas e recursos computacionais. Como citamos na aula anterior, o termo “sem servidor” não significa que não existam servidores; significa que você não precisa se preocupar com eles. A infraestrutura (provisionamento, escalonamento e manutenção de servidores) é totalmente abstraída.

O **AWS Lambda** é o serviço de computação orientado a eventos da AWS que permite executar código sem provisionar ou gerenciar servidores. Você simplesmente faz o upload do seu código como uma “função Lambda” e a AWS se encarrega de tudo o que é necessário para executá-lo e escalá-lo com alta disponibilidade. Para desenvolvedores(as) Java, isso significa focar exclusivamente na lógica da aplicação e no seu algoritmo, sem a sobrecarga de gerenciar um servidor de aplicação, uma máquina virtual ou um contêiner.

A abordagem do AWS Lambda: foco exclusivo no código

Ao deixar que a AWS se encarregue de tudo o que é necessário para executar e escalar seu código, significa que todos os seguintes pontos são completamente abstraídos:

Zero gerenciamento de servidores: não existe o conceito de “sua” máquina. Você não escolhe um tipo de instância, não instala um sistema operacional e não escreve um Dockerfile. O servidor, o contêiner e o sistema operacional são detalhes de implementação gerenciados inteiramente pela AWS.

Ambiente de execução por demanda: você apenas seleciona o runtime (ex.: “Java 21”). Quando sua função é acionada por um evento (como uma requisição HTTP), a AWS instantaneamente encontra e provisiona um ambiente de execução seguro e isolado com o Java já pronto. Seu código é carregado e executado. A AWS é responsável por manter esse runtime seguro e atualizado.

Escalabilidade e custo granular:

- **Execução sob demanda:** sua função não “existe” até que seja chamada. Se ninguém usar sua aplicação por três horas, o custo de

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

computação durante esse período é zero. Você paga apenas pelos milissegundos em que seu código está efetivamente em execução.

- **Escalabilidade massiva e automática:** se dez usuários acionam sua função simultaneamente, a AWS executa dez cópias da sua função em paralelo. Se 10.000 usuários a acionam, a AWS tenta escalar para 10.000 execuções paralelas. Essa capacidade de escalar de zero para milhares (e voltar a zero) em segundos é algo extremamente difícil e caro de se replicar em uma arquitetura tradicional.

Isso é importante porque, se aplicado corretamente, o AWS Lambda pode ser um componente central no desenho da arquitetura. Ao eliminar a necessidade de gerenciamento de infraestrutura, as equipes de desenvolvimento podem entregar novas funcionalidades e produtos com muito **mais rapidez**. O foco muda da operação para a inovação.

Além disso, o **modelo de pagamento por uso (pay-per-use) é extremamente eficiente**. Se sua função não está executando, não há custo. Isso é ideal para aplicações com cargas de trabalho variáveis ou imprevisíveis, evitando o desperdício de manter servidores ociosos.

As aplicações construídas sobre o Lambda também herdam a **escalabilidade e a tolerância a falhas** da AWS. Lidar com picos de tráfego se torna uma preocupação do provedor, não do desenvolvedor. Por fim, o Lambda é a peça central de arquiteturas de microsserviços e aplicações orientadas a eventos, **padrões cada vez mais adotados pela indústria** para construir sistemas robustos e flexíveis.

Como o AWS Lambda funciona?

O funcionamento do Lambda é baseado em um modelo orientado a eventos. Sua função é executada em resposta a um gatilho (trigger). Esses gatilhos podem ser:

- **Requisições HTTP:** através do Amazon API Gateway, sua função pode se tornar um endpoint de API REST.
- **Modificações em serviços da AWS:** upload de um arquivo em um bucket S3, uma nova mensagem em uma fila SQS ou uma alteração em uma tabela do DynamoDB.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- **Execuções agendadas:** semelhante a tarefas cron, usando o AWS EventBus.
- **Invocações diretas:** através do AWS SDK ou da AWS CLI.

Quando um gatilho é acionado, o Lambda executa sua função em um ambiente de execução isolado e pré-configurado com os recursos necessários. A AWS cuida do escalonamento, iniciando quantas instâncias da sua função forem necessárias para lidar com o volume de eventos, de forma totalmente automática. A cobrança é feita apenas pelo tempo de computação que sua função efetivamente utiliza, medido em milissegundos.

Para a pessoa que está programando em Java, uma função Lambda é, em sua essência, um pacote de deployment (um arquivo .zip ou .jar) contendo seu código compilado e suas dependências. No coração desse pacote está a classe que implementa a interface `RequestHandler<I, O>`, onde `I` é o objeto de entrada (representando o evento do gatilho) e `O` é o objeto de saída. Quando um evento aciona sua função, o serviço Lambda invoca o método `handleRequest` dessa classe.

O ambiente de execução, ou **runtime**, é muito mais do que uma simples JVM. É um microcontainer seguro e isolado, conhecido como **Firecracker**, que executa um sistema operacional Linux minimalista com o runtime Java pré-instalado. A AWS gerencia todo o ciclo de vida desse ambiente. Na primeira invocação (o **cold start**), a AWS provisiona o microcontainer, baixa seu código, inicia a JVM e executa o código estático de inicialização da sua classe. É crucial entender que qualquer código colocado em construtores estáticos ou fora do método handler será executado apenas uma vez durante o cold start, tornando-o o local ideal para inicializar recursos pesados, como clientes de SDK, conexões a bancos de dados ou a leitura de arquivos de configuração, que serão então reutilizados em invocações “quentes” subsequentes, otimizando drasticamente a performance.

Gerenciar dependências em um projeto Lambda Java requer atenção. Embora seja possível empacotar todas as dependências em um único “fat jar”, isso pode resultar em pacotes muito grandes, aumentando o tempo do cold start. Uma estratégia mais eficiente é usar **Lambda Layers**. Layers são pacotes .zip que podem conter bibliotecas, um runtime customizado ou outras dependências. Eles são publicados separadamente e podem ser compartilhados entre múltiplas funções, como uma

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

camada (ou mais) em que a função é executada, mantendo o pacote de deployment da sua função principal enxuto e ágil.

A performance, por sua vez, está diretamente ligada à configuração de memória. Aumentar a memória não apenas dá mais heap space para sua aplicação, mas também aloca proporcionalmente mais poder de CPU, o que pode reduzir significativamente o tempo de execução e, paradoxalmente, diminuir o custo total.

Para lidar com a latência do cold start em aplicações críticas, a AWS oferece a Provisioned Concurrency. Essa funcionalidade mantém um número pré-definido de ambientes de execução permanentemente inicializados e prontos para responder instantaneamente, garantindo performance consistente ao custo de não seguir mais o modelo de pagamento puramente por uso.

CONFIGURAÇÃO DO AMBIENTE DE EXECUÇÃO (RUNTIME)

Ao criar uma função Lambda, você escolhe um runtime, que é o ambiente onde seu código Java será executado. A AWS oferece runtimes gerenciados para várias versões do Java (e outras linguagens). Isso significa que a AWS é responsável por aplicar patches de segurança e atualizações ao sistema operacional e ao ambiente Java.

Você configura a quantidade de memória alocada para sua função (de 128 MB a 10 GB) e a quantidade de CPU é alocada proporcionalmente à memória. É sua responsabilidade testar e definir a alocação ideal que equilibre custo e performance. Um código de exemplo:

```
    import com.amazonaws.services.lambda.runtime.Context;
    import
com.amazonaws.services.lambda.runtime.RequestHandler;
    import com.amazonaws.services.lambda.runtime.LambdaLogger;

    public class HelloRequest {
        private String nome;

        // Construtor vazio necessário para deserialização
        public HelloRequest() {}
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
        public HelloRequest(String nome) {
            this.nome = nome;
        }

        public String getNome() {
            return nome;
        }

        public void setNome(String nome) {
            this.nome = nome;
        }
    }

    public class HelloResponse {
        private String mensagem;

        // Construtor vazio necessário para serialização
        public HelloResponse() {}

        public HelloResponse(String mensagem) {
            this.mensagem = mensagem;
        }

        public String getMensagem() {
            return mensagem;
        }

        public void setMensagem(String mensagem) {
            this.mensagem = mensagem;
        }
    }

    public class HelloLambdaFunction implements
RequestHandler<HelloRequest, HelloResponse> {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
        @Override
        public HelloResponse handleRequest(HelloRequest
request, Context context) {
            LambdaLogger logger = context.getLogger();
            logger.log("Processando requisição: " + request);

            String nome = request.getNome() != null ?
request.getNome() : "Mundo";
            String mensagem = "Olá, " + nome + "!";

            logger.log("Mensagem gerada: " + mensagem);

            return new HelloResponse(mensagem);
        }
    }
```

Código-fonte 1 – Exemplo
Fonte: elaborado pelo autor (2025)

O código acima demonstra alguns pontos importantes. O ponto de entrada, **HelloLambdaFunction**, implementa a interface requerida e que será a classe instanciada no início da execução. São definidas duas classes, HelloRequest e HelloResponse, que definem os dados de entrada e saída. Essa nomenclatura, inclusive, é semelhante à definida no protocolo HTTP, mas, no caso do lambda, vamos receber dados de diferentes origens (gatilhos).

Vemos também um **objeto de contexto**, que pode ser usado para obter informações da execução da função, como data de início, runtime e outros dados importantes. Nesse objeto, também podemos obter um objeto de log. Por fim, vemos que o retorno é sempre uma instância da classe definida como **Response**.

Gerenciamento do ciclo de vida

O ciclo de vida de um ambiente de execução Lambda possui três fases principais:

Init (Inicialização/provisionamento): esta fase acontece na primeira vez que sua função é invocada (o cold start) ou após uma atualização. Nela, a AWS inicia o ambiente de execução, baixa o seu código e inicializa o runtime do Java e o código

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

da sua função que está fora do método handler principal (o código estático). Esta é uma oportunidade para inicializar conexões com bancos de dados, carregar configurações ou instanciar objetos pesados que podem ser reutilizados em invocações subsequentes.

Invoke (Invocação): esta é a fase onde o método handler da sua função é executado para processar o evento de gatilho. O mesmo ambiente de execução pode ser reutilizado para múltiplas invocações, o que é conhecido como uma execução “quente” (warm start). Isso torna as invocações subsequentes muito mais rápidas, pois a fase Init é pulada.

Shutdown (Desligamento): se o ambiente de execução permanecer ocioso por um período, a AWS o desativará. O Lambda envia um evento de desligamento ao runtime, permitindo que o código realize tarefas de limpeza, como fechar conexões de banco de dados de forma graciosa.

Compreender e otimizar a fase Init é a chave para mitigar os cold starts e construir aplicações serverless eficientes e de alto desempenho na AWS.

O AWS Lambda como componente em uma arquitetura cloud

O AWS Lambda raramente opera isoladamente. Ele une diversos serviços da AWS em arquiteturas robustas e orientadas a eventos. Compreender os padrões de design é fundamental. Por exemplo, em uma API RESTful, o **Amazon API Gateway** atua como o trigger, recebendo requisições HTTP e as traduzindo em eventos que invocam funções Lambda específicas para cada rota. Para processos assíncronos e desacoplados, uma função Lambda pode publicar mensagens em um tópico do **Amazon SNS** ou em uma fila do **Amazon SQS**, que por sua vez podem acionar outras funções Lambda. Esse padrão de “fan-out” é incrivelmente poderoso para processamento paralelo.

Para orquestrar fluxos de trabalho complexos com múltiplos passos, lógica condicional e tratamento de erros, o **AWS Step Functions** permite definir uma máquina de estados visual que coordena a execução de várias funções Lambda, transformando uma série de tarefas simples em um processo de negócio sofisticado e resiliente, sem a necessidade de escrever código de orquestração complexo.

Em uma arquitetura monolítica, depurar um problema pode ser tão simples quanto analisar um arquivo de log. No mundo distribuído e efêmero do serverless, a

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

observabilidade se torna um desafio de primeira ordem, cujo pilar é o **Amazon CloudWatch**, que integra-se nativamente ao Lambda.

Toda a saída padrão (`System.out.println`) do seu código Java é automaticamente enviada para o CloudWatch Logs, onde você pode pesquisar e analisar os logs de execução. O CloudWatch Metrics, por sua vez, coleta métricas vitais sobre suas funções, como o número de invocações, duração, erros e concorrência.

No entanto, para entender o fluxo completo de uma requisição que atravessa múltiplos serviços (API Gateway > Lambda > DynamoDB > SQS > Lambda), o **AWS X-Ray** é indispensável. Ao integrar o X-Ray SDK ao seu código Java, você pode gerar "traces" que fornecem um mapa visual completo da jornada da requisição, identificando gargalos de performance e pontos de falha em todo o sistema distribuído. Dominar essas ferramentas não é opcional; é um requisito para manter e operar aplicações serverless em produção de forma eficaz.

Desafios da arquitetura e uso do Serverless na AWS

Apesar das vantagens, a adoção de uma arquitetura serverless com AWS Lambda apresenta seus próprios **desafios de uso**:

• **Cold Starts:** já citamos anteriormente, mas é interessante retomar este ponto: uma função que nunca foi executada ou que não é executada por um tempo pode ser desativada/descomissionada. A primeira requisição para essa função "fria" pode ter uma latência maior, pois o ambiente de execução precisa ser inicializado. Para aplicações sensíveis à latência, como APIs interativas, isso pode ser um problema. Existem estratégias para mitigar isso, como o Provisioned Concurrency, mas exigem configuração adicional.

• **Limites de execução:** as funções Lambda têm um tempo máximo de execução (atualmente 15 minutos). Para processos de longa duração, é necessário repensar a arquitetura, quebrando a tarefa em funções menores e orquestrando-as, por exemplo, com o AWS Step Functions.

• **Monitoramento e depuração:** diagnosticar problemas pode ser mais complexo. Como você não tem acesso ao servidor, a observabilidade depende de ferramentas de logging e monitoramento, como o Amazon CloudWatch. Rastrear uma

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

requisição que passa por múltiplas funções Lambda exige o uso de serviços como o AWS X-Ray.

É importante também pensar nos desafios que o desenho e a implementação de uma arquitetura serverless na AWS nos apresenta. Os principais **desafios da arquitetura** são:

Complexidade de gerenciamento: uma aplicação serverless complexa pode ser composta por dezenas ou centenas de funções Lambda. Gerenciar as permissões, configurações e o deploy de todas essas funções pode se tornar um desafio. Ferramentas como o AWS SAM (Serverless Application Model) ou o Serverless Framework são essenciais para gerenciar a aplicação como um todo.

Design da função: decidir a granularidade de uma função é um desafio arquitetural. Criar funções muito pequenas e com responsabilidade única pode levar a uma explosão no número de funções e a uma complexidade na orquestração. Por outro lado, criar “Lambda-liths”, ou “micro-litos” (funções monolíticas) pode anular muitos dos benefícios do serverless, como a configuração granular de recursos e permissões.

Gerenciamento de estado: funções Lambda são, por natureza, stateless (sem estado). Qualquer estado necessário entre execuções deve ser persistido em um serviço externo, como um banco de dados (DynamoDB, RDS) ou um sistema de cache (ElastiCache).

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

A abordagem serverless na AWS implementa as ideias básicas de arquitetura serverless, transferindo o gerenciamento de servidores para a nuvem e permitindo que pessoas desenvolvedoras foquem na codificação. No caso da AWS, focamos em usar o serviço AWS Lambda, que é a solução de função como serviço.

Vimos um exemplo de código que nos ajuda a entender que a complexidade de desenvolvimento de funções é muito baixa, sendo uma barreira de entrada quase inexistente. Entendemos, por fim, alguns desafios de uso de funções na AWS, com suas particularidades.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

AWS. **AWS Well-Architected**. Disponível em https://aws.amazon.com/pt/architecture/well-architected. Acesso em: 07 jul. 2025.

INDRASIRI, K; SUHOTHAYAN, S. **Design Patterns for Cloud Native Applications**. [S.l.]: O'Reilly, 2021.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. [S.l.]: O'Reilly, 2021.

FIAP

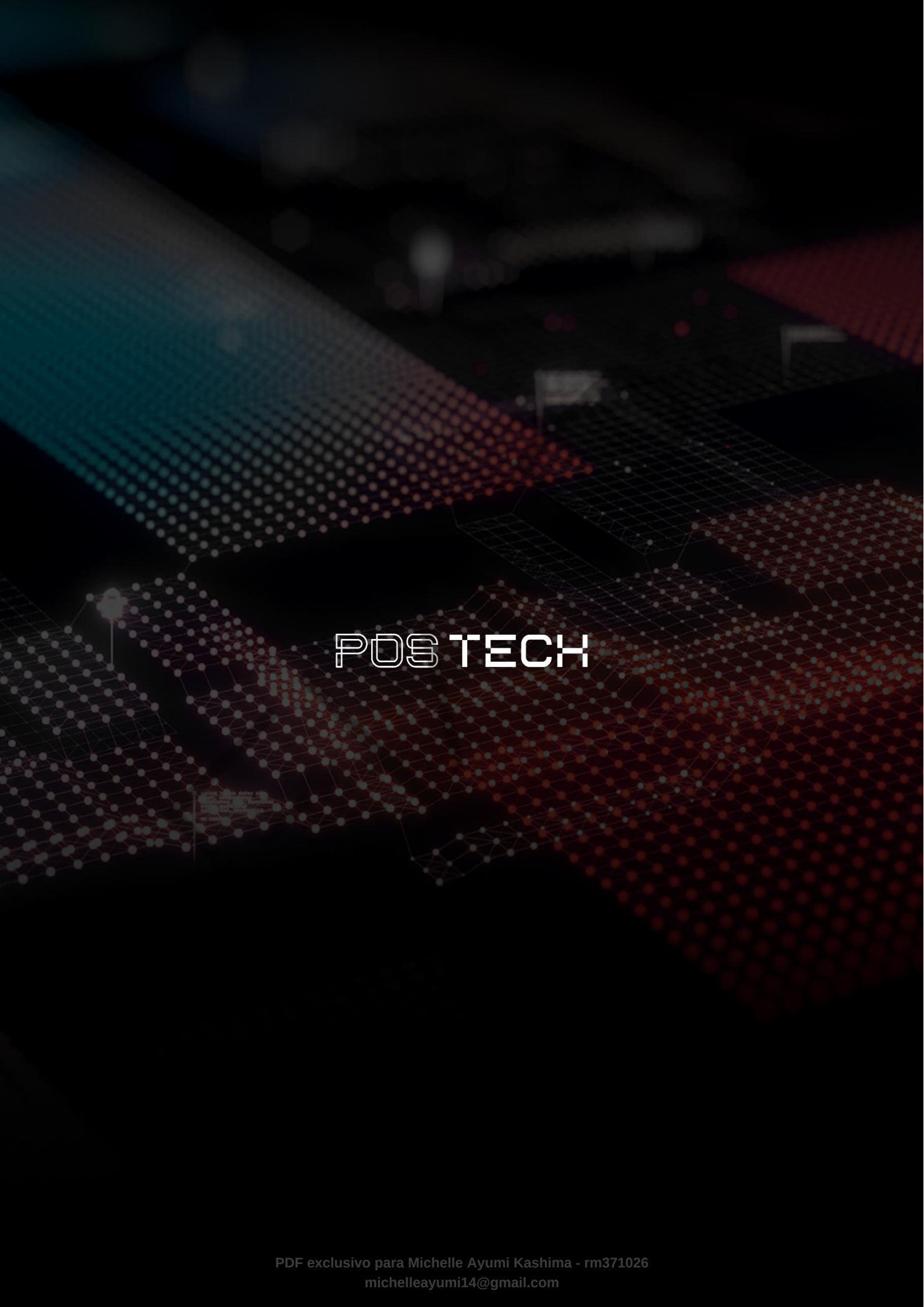
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Serverless. AWS Lambda.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com